RUNBOOK DE EXECUÇÃO

Real-Time Payment Fraud Pipeline

Checklist operacional baseado na transcrição do vídeo do instrutor

Finalidade	Servir como roteiro de execução e ponto de restauração de contexto entre conversas.
Fonte principal	Arquivo SRT enviado pelo usuário — demonstração fim a fim de aproximadamente 15min37s.
Adaptação do projeto	O instrutor demonstra Neon; nosso projeto utiliza Aiven e Terraform. A equivalência deve ser feita sem alterar o fluxo funcional demonstrado.

1. Como este documento deve ser utilizado

Este checklist representa a sequência funcional mostrada pelo instrutor. Cada etapa possui um gate: não avançar enquanto o critério de conclusão não estiver comprovado. O objetivo não é reproduzir a interface ou os comandos literalmente, mas concluir as mesmas entregas no nosso ambiente, mantendo Terraform como fonte de verdade para infraestrutura.

- Marcar uma atividade como concluída somente após executar e observar o resultado esperado.
- Distinguir IMPLEMENTADO (há código), EXECUTADO (foi aplicado), VALIDADO (há evidência funcional) e PENDENTE.
- Não tratar Flink executando isoladamente como prova de que o CDC está entregando eventos.
- Não alterar o ciclo de criação/destruição do conector Terraform que já foi validado.
- Não imprimir tokens, senhas, API secrets, connection strings ou credenciais no terminal/logs.
- Usar uma sessão de terminal controlada para as variáveis de ambiente; o instrutor mantém as credenciais em variáveis durante a execução.
- No nosso projeto, operações com custo ou recursos remotos devem ocorrer somente na janela planejada e com teardown previsto.

2. Visão macro da sequência do instrutor

Tempo aproximado	Atividade principal
00:00–03:22	Environment, cluster, identidades, API Keys e ACLs
03:22–05:05	Projeto de banco, replicação lógica, connection string e carga inicial
05:05–06:40	Conferência dos dados, credencial CDC e identidade/API Key do Kafka Connect
06:40–07:45	Criação e execução do conector CDC; geração dos tópicos
07:45–09:12	Consumo dos tópicos e validação de INSERT/DELETE e before/after
09:12–09:37	Contrato/schema e validação de compatibilidade
09:37–09:58	Criação do Flink Compute Pool
09:58–10:40	Workspace SQL e ajuste de chave primária
10:40–11:28	Enriquecimento com temporal join e rowtime
11:28–12:51	Detecção de padrão com MATCH_RECOGNIZE
12:51–13:43	Execução contínua, consulta de inspeção e leitura dos resultados
13:43–14:20	Ingestão sequencial de cenários e geração de alerta
14:20–15:17	Observabilidade, segurança e custo/billing
15:17–15:37	Resumo e encerramento

3. Checklist detalhado por etapa

Etapa 1 — Environment, cluster, identidades e credenciais | 00:00–03:22

Objetivo: Criar ou selecionar o ambiente Confluent, o cluster Kafka e as identidades que permitirão operar Kafka e Schema Registry.

Atividades do instrutor / execução

- ☐ Criar o Environment e o cluster Kafka (o instrutor começa com recursos vazios).
- ☐ Conectar ao cluster e conferir que ele está vazio antes da carga.
- ☐ Criar/selecionar a identidade de serviço de aplicação/CDC e conceder acesso ao cluster.
- ☐ Gerar API Key/Secret para o cluster Kafka e conceder os ACLs necessários.
- ☐ Criar/selecionar a identidade do Schema Registry e gerar sua API Key/Secret.
- ☐ Guardar IDs, endpoints e credenciais em variáveis de ambiente da sessão; não copiar segredos para o código.
- ☐ Manter o terminal que contém as variáveis aberto durante a execução.

O que precisamos ter para executar

- ☐ Acesso administrativo à organização Confluent.
- ☐ Environment ID, Kafka Cluster ID e bootstrap endpoint.
- ☐ Identidades de serviço e permissões para Kafka/Schema Registry.
- ☐ API Key/Secret de Kafka e API Key/Secret do Schema Registry.
- ☐ Mecanismo seguro para carregar variáveis no PowerShell.

Critério de conclusão (gate)

- ☐ Environment e cluster corretos identificados.
- ☐ Identidades e ACLs criados/conferidos.
- ☐ Credenciais testadas sem expor seus valores.
- ☐ Sessão de execução pronta para as próximas etapas.

Evidências a registrar

- ☐ IDs do Environment e cluster (não secretos).
- ☐ Lista de identidades e ACLs concedidos.
- ☐ Resultado booleano de presença das variáveis/credenciais, nunca os valores.

Atenção / observação: O vídeo demonstra criação manual/CLI. No nosso projeto, recursos já existentes devem ser reconciliados com Terraform; não recriar o que já existe.

Etapa 2 — Preparação do banco e habilitação de replicação lógica | 03:22–05:05

Objetivo: Preparar o banco de origem para aceitar o conector CDC e carregar as tabelas/dados iniciais.

Atividades do instrutor / execução

- ☐ Criar o projeto/banco de dados (no vídeo, Neon).
- ☐ Habilitar Logical Replication nas configurações do projeto.
- ☐ Copiar a connection string e configurá-la no ambiente de execução.
- ☐ Confirmar que o Docker está ativo, conforme a demonstração.
- ☐ Executar o SQL de criação das tabelas e carga inicial.
- ☐ Conferir que os dados foram carregados no schema payments.

O que precisamos ter para executar

- ☐ Acesso administrativo ao banco de origem.
- ☐ Host, porta, database, usuário e conexão TLS.
- ☐ SQL de DDL e seed fornecido/preparado para o desafio.
- ☐ Confirmação de que o serviço/provedor permite CDC por replicação lógica.
- ☐ No nosso caso: acesso ao Aiven e credenciais válidas, sem expô-las.

Critério de conclusão (gate)

- ☐ Replicação lógica habilitada e comprovada no provedor.
- ☐ Connection string válida para a sessão de trabalho.
- ☐ Tabelas do schema esperado criadas e carga inicial concluída.

Evidências a registrar

- ☐ Evidência da configuração de replicação lógica habilitada.
- ☐ Saída de validação de conexão sem imprimir connection string.
- ☐ Contagens das tabelas após a carga.

Atenção / observação: O instrutor usa Neon. Nosso banco é Aiven. Não copiar instruções de interface do Neon literalmente: precisamos verificar a configuração equivalente no Aiven e validar o que o conector exige.

Etapa 3 — Conferência da carga, usuário CDC e identidade do Kafka Connect | 05:05–06:40

Objetivo: Confirmar o volume de dados e preparar a autenticação usada pelo conector.

Atividades do instrutor / execução

- ☐ Consultar as contagens das entidades carregadas.
- ☐ O vídeo apresenta como referência: 200 clientes, 240 contas, 320 cartões, 360 estabelecimentos/merchants e 2.000 transações.
- ☐ Executar o SQL de preparação/alteração da senha do usuário CDC.
- ☐ Definir uma senha forte para o usuário CDC.
- ☐ Criar a identidade/API Key destinada ao Kafka Connect (o instrutor também a chama de CDC Writer).

O que precisamos ter para executar

- ☐ SQL de conferência das tabelas.
- ☐ Usuário com privilégio administrativo suficiente para preparar o usuário CDC.
- ☐ Definição dos privilégios que o conector precisa no PostgreSQL.
- ☐ Identidade e credencial Kafka Connect no Confluent.
- ☐ Armazenamento seguro da senha e da API Key/Secret.

Critério de conclusão (gate)

- ☐ Contagens conferidas ou divergências explicadas.
- ☐ Usuário CDC preparado com os privilégios necessários.
- ☐ Credencial Kafka Connect criada e disponível ao Terraform/conector.

Evidências a registrar

- ☐ Contagens por tabela.
- ☐ Confirmação de existência/estado do usuário CDC sem exibir senha.
- ☐ Confirmação de presença da credencial do Kafka Connect sem exibir o segredo.

Atenção / observação: Os números são os valores falados pelo instrutor e funcionam como referência da carga demonstrada. Se nosso dataset tiver volumes diferentes, registrar a diferença; não fabricar registros apenas para igualar números sem verificar o SQL-fonte.

Etapa 4 — Construção e execução do conector CDC | 06:40–07:45

Objetivo: Conectar PostgreSQL ao Kafka e fazer os eventos das tabelas selecionadas chegarem aos tópicos.

Atividades do instrutor / execução

- ☐ Selecionar as tabelas de origem que serão capturadas (o áudio menciona transactions, accounts e cards; a enumeração está parcialmente redundante na transcrição).
- ☐ Gerar o arquivo/configuração do conector.
- ☐ Executar a criação do conector.
- ☐ O instrutor gera o arquivo para executar e depois o remove do repositório para não acumular artefatos temporários.
- ☐ Aguardar a propagação/criação dos tópicos (o instrutor comenta que pode levar cerca de dois minutos).
- ☐ Conferir o status RUNNING do conector e a existência dos tópicos.

O que precisamos ter para executar

- ☐ Banco preparado e usuário CDC funcional.
- ☐ Host/porta/database/schema e credencial CDC.
- ☐ Kafka Connect API Key/Secret e cluster de destino.
- ☐ Configuração Debezium compatível com o banco e com as tabelas.
- ☐ Terraform e lifecycle scripts já existentes no nosso repositório.

Critério de conclusão (gate)

- ☐ Conector criado pelo mecanismo Terraform já existente.

- ☐ Estado do conector RUNNING.
- ☐ Tópicos CDC esperados criados no Kafka.

Evidências a registrar

- ☐ Evidência do Terraform apply bem-sucedido.
- ☐ Status do conector.
- ☐ Lista de tópicos gerados.
- ☐ Evidência de teardown planejado para encerrar o recurso faturável após a janela de teste.

Atenção / observação: Regra do nosso projeto: preservar o ciclo Terraform de criação/destruição que já funciona. Não substituir por criação manual nem reescrever o recurso sem uma falha comprovada.

Etapa 5 — Consumo dos tópicos e validação do CDC | 07:45–09:12

Objetivo: Provar que alterações no PostgreSQL são publicadas no Kafka e que o conteúdo representa corretamente as operações.

Atividades do instrutor / execução

- ☐ Executar um consumidor simples para ler os tópicos.
- ☐ Confirmar que existem eventos nos tópicos.
- ☐ Executar uma inserção e uma exclusão na origem.
- ☐ Inspecionar os eventos resultantes e os campos before/after quando aplicáveis.
- ☐ Confirmar que o evento de exclusão aparece com a semântica esperada.

O que precisamos ter para executar

- ☐ Conector RUNNING.
- ☐ Tópicos CDC criados.
- ☐ Credencial Kafka com permissão de consumo.
- ☐ Cliente/CLI ou consumidor usado para ler os tópicos.
- ☐ SQL controlado de INSERT e DELETE em registros de teste.

Critério de conclusão (gate)

- ☐ Uma alteração feita no PostgreSQL é observada no tópico correspondente.
- ☐ O evento de INSERT é identificado.
- ☐ O evento de DELETE é identificado e seu payload é compreendido.
- ☐ O consumidor consegue ler os eventos sem erro de autenticação/serialização.

Evidências a registrar

- ☐ Saída sanitizada do consumidor com operação e chave do registro.
- ☐ Evidência do INSERT e do DELETE correlacionados aos testes.
- ☐ Sem imprimir credenciais ou dados pessoais desnecessários.

Atenção / observação: Este é o gate mais importante antes do Flink: autenticação do conector ou status RUNNING, isoladamente, não comprovam que os eventos estão chegando corretamente.

Etapa 6 — Contrato/schema e compatibilidade | 09:12–09:37

Objetivo: Habilitar/validar o contrato de dados no Schema Registry e testar uma alteração de schema.

Atividades do instrutor / execução

- ☐ Ativar o contrato/schema para o fluxo.
- ☐ Adicionar/testar um campo obrigatório sem valor default, conforme a demonstração.
- ☐ Executar a validação de compatibilidade.
- ☐ Confirmar que a alteração testada não quebra o contrato existente.

O que precisamos ter para executar

- ☐ Schema Registry acessível.
- ☐ API Key/Secret do Schema Registry.
- ☐ Subjects e schemas dos tópicos envolvidos.
- ☐ Regra de compatibilidade configurada para o subject.
- ☐ Versão do schema que será validada.

Critério de conclusão (gate)

- ☐ Schema registrado/validado.
- ☐ Teste de compatibilidade concluído.
- ☐ Resultado da validação entendido e registrado.

Evidências a registrar

- ☐ Subject e versão do schema.
- ☐ Resultado da validação de compatibilidade.
- ☐ Diff da evolução de schema, sem credenciais.

Atenção / observação: A transcrição não informa o modo exato de compatibilidade nem reproduz o schema completo. Não assumir BACKWARD/FULL ou nomes de campos sem consultar os arquivos do projeto.

Etapa 7 — Criação do Flink Compute Pool | 09:37–09:58

Objetivo: Disponibilizar capacidade de processamento Flink para executar as consultas SQL.

Atividades do instrutor / execução

- ☐ Criar o Compute Pool.
- ☐ Selecionar o Environment/cluster correspondente.
- ☐ Abrir o cluster e localizar o Compute Pool criado.

O que precisamos ter para executar

- ☐ Environment e cluster Confluent corretos.
- ☐ Região compatível com os recursos Kafka/Schema Registry.
- ☐ Permissões para criar/usar Flink Compute Pool.
- ☐ Capacidade/custo planejados para a janela de execução.

Critério de conclusão (gate)

- ☐ Compute Pool provisionado e selecionável no ambiente.
- ☐ Workspace Flink acessível.

Evidências a registrar

- ☐ ID/nome/região do Compute Pool.
- ☐ Estado de provisionamento.
- ☐ Evidência do custo/capacidade configurada.

Atenção / observação: No nosso projeto, o Compute Pool já existe e o Flink já foi executado isoladamente. Esta etapa não deve ser refeita sem necessidade.

Etapa 8 — Workspace SQL e chave primária | 09:58–10:40

Objetivo: Preparar a estrutura/tabela que será usada nas transformações Flink.

Atividades do instrutor / execução

- ☐ Abrir o SQL Workspace associado ao Compute Pool.
- ☐ Executar ALTER TABLE para adicionar account_id como chave primária, conforme a demonstração.
- ☐ O instrutor comenta que precisou executar as alterações uma de cada vez.

O que precisamos ter para executar

- ☐ Compute Pool disponível.
- ☐ Tabelas Kafka/SQL já reconhecidas pelo Flink.
- ☐ Definição atual da tabela e metadados de chave.
- ☐ SQL DDL correto para o objeto do projeto.

Critério de conclusão (gate)

- ☐ Alteração de tabela aceita pelo Flink.
- ☐ Chave primária reconhecida no objeto esperado.
- ☐ Nenhum statement concorrente conflitante.

Evidências a registrar

- ☐ SQL DDL executado.
- ☐ Resultado de sucesso.
- ☐ Definição final da tabela.

Atenção / observação: A transcrição fala em account_id como chave primária, mas não contém o DDL completo. Conferir o esquema real antes de reproduzir.

Etapa 9 — Enriquecimento temporal | 10:40–11:28

Objetivo: Combinar os dados de transações com dados de contas por meio de temporal join.

Atividades do instrutor / execução

- ☐ Criar uma consulta de enriquecimento usando JOIN.
- ☐ Utilizar temporal join, conforme explicado pelo instrutor.
- ☐ Definir o tempo de referência da consulta com rowtime/tempo do sistema (o áudio transcreve o trecho SQL de forma imprecisa).
- ☐ Executar a consulta e confirmar que o enriquecimento é aceito.

O que precisamos ter para executar

- ☐ Tabelas de transações e contas acessíveis no Flink.
- ☐ Chaves de relacionamento identificadas.
- ☐ Coluna temporal/rowtime corretamente definida.
- ☐ DDL e SQL de enriquecimento.

Critério de conclusão (gate)

- ☐ Temporal join compilado e executado.
- ☐ Resultado contém os atributos esperados da transação e da conta.
- ☐ Sem multiplicação indevida de registros ou perda inexplicada.

Evidências a registrar

- ☐ SQL do temporal join.
- ☐ Estado do statement.
- ☐ Amostra sanitizada do resultado enriquecido.

Atenção / observação: A expressão ouvida como “front system” pode ser uma referência a FOR SYSTEM_TIME. A transcrição não permite garantir a sintaxe exata; validar no SQL original do instrutor/projeto.

Etapa 10 — Detecção de padrões com MATCH_RECOGNIZE | 11:28–12:51

Objetivo: Analisar a sequência de transações por cartão e identificar um padrão dentro de uma janela de 60 segundos.

Atividades do instrutor / execução

- ☐ Consultar a tabela enriquecida.
- ☐ Aplicar MATCH_RECOGNIZE.
- ☐ Particionar por card_id.
- ☐ Ordenar por rowtime.
- ☐ Definir medidas para o primeiro e o último rowtime da sequência.
- ☐ Avaliar a sequência transação a transação.
- ☐ Restringir o padrão ao intervalo de 60 segundos.
- ☐ Aplicar a condição amount > 0.
- ☐ O instrutor descreve que, após encontrar correspondência, o processamento avança para o último elemento do padrão.

O que precisamos ter para executar

- ☐ Temporal join funcional.
- ☐ card_id disponível e não nulo nos registros elegíveis.
- ☐ rowtime definido e ordenável.
- ☐ amount disponível e numérico.
- ☐ SQL MATCH_RECOGNIZE completo e compatível com o Flink utilizado.

Critério de conclusão (gate)

- ☐ Statement MATCH_RECOGNIZE aceito pelo Flink.
- ☐ Padrão particionado por cartão e ordenado temporalmente.
- ☐ Regra de 60 segundos aplicada.
- ☐ Condição amount > 0 aplicada.
- ☐ Resultado de correspondência observável.

Evidências a registrar

- ☐ SQL integral executado.
- ☐ Estado do statement.
- ☐ Linhas de resultado com card_id e tempos inicial/final.
- ☐ Cenários positivos e negativos documentados.

Atenção / observação: A transcrição tem um trecho ambíguo sobre “terceira casa decimal” e não fornece o padrão completo, aliases ou DEFINE. Não inventar a expressão; usar o SQL efetivo do desafio como fonte.

Etapa 11 — Execução contínua e inspeção dos resultados | 12:51–13:43

Objetivo: Manter o processamento contínuo e consultar separadamente a tabela/resultado para observar os registros.

Atividades do instrutor / execução

- ☐ Distinguir o statement contínuo do statement de consulta não contínua.
- ☐ Abrir outra aba/consulta para inspecionar a tabela criada.
- ☐ Executar SELECT e aguardar a chegada/carregamento dos card IDs.
- ☐ Observar registros sem cartão e compreender como são tratados.

O que precisamos ter para executar

- ☐ Statements das etapas de enriquecimento e MATCH_RECOGNIZE.
- ☐ Compute Pool ativo.
- ☐ Permissões de leitura dos resultados.
- ☐ Cliente de consulta/consumidor de resultados.

Critério de conclusão (gate)

- ☐ Statement contínuo permanece executando.
- ☐ Consulta de inspeção retorna registros.
- ☐ Comportamento de registros sem card_id é conhecido.

Evidências a registrar

- ☐ Estado do statement contínuo.
- ☐ Saída sanitizada do SELECT.
- ☐ Critério de tratamento de card_id nulo.

Atenção / observação: Não confundir ausência de linhas imediatas com falha de compilação. A consulta contínua depende de eventos e a inspeção deve ser feita em separado.

Etapa 12 — Cenários de teste e geração de alertas | 13:43–14:20

Objetivo: Produzir eventos de teste em sequência para verificar o comportamento da regra e gerar um alerta quando houver fraude.

Atividades do instrutor / execução

- ☐ Executar os comandos de ingestão sequencial disponibilizados no roteiro.
- ☐ Gerar uma transação legítima/inocente.
- ☐ Aplicar/observar o watermark mencionado pelo instrutor.
- ☐ Gerar uma sequência considerada fraudulenta.
- ☐ Verificar a saída/alerta no destino indicado pelo desafio (o áudio transcreve o nome como “CPL”, sem clareza suficiente para identificar o destino).

O que precisamos ter para executar

- ☐ SQL/script de cenários de teste.
- ☐ Pipeline CDC e Kafka comprovadamente funcionando.
- ☐ Enriquecimento e MATCH_RECOGNIZE executando.
- ☐ Watermark configurado.
- ☐ Destino de alertas e contrato de saída definidos.

Critério de conclusão (gate)

- ☐ Cenário legítimo não gera alerta indevido.
- ☐ Cenário fraudulento satisfaz a regra.
- ☐ Alerta aparece no destino esperado.

- ☐ Dados do alerta permitem rastrear o conjunto de transações que o originou.

Evidências a registrar

- ☐ Comandos/IDs dos registros de teste.
- ☐ Resultado esperado versus observado.
- ☐ Mensagem de alerta recebida no destino.
- ☐ Evidência dos cenários positivos e negativos.

Atenção / observação: O vídeo não detalha o payload do alerta nem permite identificar com segurança o termo transcrito como “CPL”. Confirmar o destino no material SQL/configuração do desafio.

Etapa 13 — Observabilidade, segurança e custos | 14:20–15:17

Objetivo: Inspecionar o ambiente e demonstrar que o desafio foi executado com visibilidade sobre recursos, segurança e billing.

Atividades do instrutor / execução

- ☐ Consultar o cluster e seu tempo/horário de criação.
- ☐ Inspecionar os recursos/identidades de segurança existentes no cluster.
- ☐ Consultar o custo do desafio e a utilização/billing da operação.

O que precisamos ter para executar

- ☐ Acesso de leitura à administração Confluent.
- ☐ Inventário de clusters, Compute Pools, identidades e permissões.
- ☐ Dados de billing/uso disponíveis para a conta.

Critério de conclusão (gate)

- ☐ Recursos utilizados identificados.
- ☐ Identidades e permissões revisadas.
- ☐ Consumo/custo observado e registrado.

Evidências a registrar

- ☐ Inventário final de recursos.
- ☐ Resumo de identidades/permissões.
- ☐ Captura ou exportação dos dados de billing/uso.

Atenção / observação: O instrutor demonstra observação; não apresenta no trecho transcrito uma política completa de segurança, um orçamento-alvo ou um procedimento de teardown.

4. Adaptação obrigatória: demonstração do instrutor × nosso projeto

O vídeo é a referência do fluxo funcional, mas utiliza Neon e demonstra várias ações via CLI/UI. Nosso projeto utiliza Aiven e Terraform. A adaptação deve preservar as entregas e os gates, não copiar literalmente a ferramenta.

Componente	Instrutor	Nosso projeto	Diretriz de adaptação
Banco de origem	Neon	Aiven PostgreSQL	Verificar no Aiven a habilitação/capacidade de logical replication e os privilégios necessários ao CDC.
Provisionamento	Criação demonstrada na CLI/UI	Terraform	Manter Terraform como fonte de verdade e reconciliar os recursos existentes.
Conector	Arquivo temporário executado e removido	Recurso Terraform existente	Preservar a implementação já funcional de create/destroy; não reescrever sem evidência de defeito.
Kafka	Cluster Confluent	Cluster Confluent existente	Comprovar tópicos e eventos reais antes de avançar.
Flink	Compute Pool e SQL Workspace	Flink existente	O statement já executou isoladamente; falta provar a chegada dos eventos pelo CDC.
Credenciais	Variáveis na sessão de terminal	Segredos protegidos/variáveis de ambiente	Nunca imprimir segredos em logs ou terminal.

5. Estado conhecido do nosso projeto (contexto de continuidade)

Estes pontos vêm do histórico de trabalho e das confirmações do usuário; não são afirmações feitas pelo instrutor:

☐ O conector CDC já possui implementação Terraform que cria e destrói automaticamente. Preservar esse comportamento.
☐ O Flink já executou statements e a regra foi trabalhada; ainda não está comprovado o fluxo real de eventos passando pelo conector.
☐ O conector não deve ser tratado como funcional ponta a ponta apenas porque o Terraform consegue criá-lo.
☐ O foco imediato é acessar o Aiven e preparar corretamente o PostgreSQL para CDC; somente depois construir/executar uma nova janela do conector.
☐ Após comprovar CDC → Kafka, avançar para a integração Flink e validar a regra fim a fim.
☐ O conector gerenciado é faturável enquanto existe; executar em janela controlada e destruir após o teste.
☐ Todos os checkpoints incrementais permanecem na série V1. A série V2 fica reservada à futura reconstrução integral via Terraform.

6. Checklist de retomada — onde estamos e o que fazer agora

☐ Acessar o Aiven com autenticação válida (o histórico registrou token expirado/401).
☐ Inspecionar a configuração do serviço PostgreSQL Aiven relacionada a logical replication/CDC.
☐ Confirmar usuário CDC, privilégios, database, schema e tabelas que o conector precisa capturar.
☐ Confirmar a publicação/slot de replicação e seu ciclo de vida, sem deixar slots órfãos após o teardown.
☐ Validar que o SQL de origem e os dados necessários existem no Aiven.
☐ Executar um teste controlado de alteração no PostgreSQL e confirmar os pré-requisitos para o conector.
☐ Somente então executar o ciclo Terraform existente para criar o conector.
☐ Comprovar que INSERT/UPDATE/DELETE aparecem nos tópicos Kafka.
☐ Destruir o conector pelo mecanismo Terraform já existente após a janela de validação.
☐ Depois da comprovação CDC → Kafka, executar o enriquecimento temporal e MATCH_RECOGNIZE no Flink.
☐ Executar cenários positivos/negativos e comprovar o alerta no tópico/destino correto.
☐ Registrar observabilidade, segurança, billing, documentação e teardown.

7. Regras para não perder o rumo

- Não começar pelo Flink quando o CDC ainda não foi validado.
- Não alterar SQL Flink apenas porque não chegaram eventos; primeiro provar o caminho PostgreSQL → conector → Kafka.
- Não substituir ou reescrever o Terraform do conector que já cria e destrói corretamente.
- Não confundir recursos existentes no State com serviços necessariamente ativos; conferir o estado remoto quando isso for relevante.
- Não executar ações manuais fora do Terraform para recursos que já são gerenciados por ele, salvo justificativa técnica explícita.
- Não avançar de etapa sem critério de aceite e evidência.
- Não imprimir secrets, tokens, senhas, connection strings ou conteúdo de arquivos de credenciais.
- Ao encontrar erro, corrigir a etapa atual; não reabrir etapas já validadas sem evidência de regressão.
- Foco em executar e concluir o desafio, não em criar diagnósticos paralelos sem necessidade.

8. Limitações da transcrição

A transcrição automática contém trechos com reconhecimento impreciso de termos técnicos. Por isso, este checklist registra a intenção audível sem inventar SQL, nomes de campos ou configurações que não aparecem com clareza.

- A enumeração das tabelas do conector entre 06:40 e 07:03 contém repetição/ambiguidade. Confirmar a lista exata no arquivo de configuração do instrutor ou no enunciado.

- O modo de compatibilidade do Schema Registry e o schema completo não são ditos no trecho.
- O DDL do ALTER TABLE e a definição da chave primária não aparecem integralmente.
- A expressão exata do temporal join/rowtime não pode ser recuperada apenas pelo texto transcrito.
- O padrão MATCH_RECOGNIZE, suas variáveis, DEFINE e cláusula de skip não são reproduzidos integralmente.
- O termo do destino do alerta em 13:43–14:02 foi transcrito como “CPL”; confirmar no material original.
- O vídeo não detalha o conteúdo completo do teardown final.

9. Registro de execução

Use este quadro para registrar o progresso real. Uma etapa só muda para VALIDADA quando houver evidência.

Etapa	Status	Data	Evidência/arquivo	Observação ou bloqueio
01				
02				
03				
04				
05				
06				
07				
08				
09				
10				
11				
12				
13				